Multi-step Genetic Algorithm for Solving Dynamic Flexible Job Shop Scheduling Problem

Guiying Ning
Mathematical Physics Teaching Department
Liuzhou Institute of Technology
Liuzhou, China
E-mail: guiyingning@126.com

Dunqian Cao
College of Science
Guangxi University for Nationalities
Nanning, China

2021 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA) | 978-1-6654-3561-1/21/$31.00 ©2021 IEEE | DOI: 10.1109/AEECA52519.2021.9574127

***Abstract*—Aiming at the problem of dynamic flexible job shop scheduling, a multi-step genetic algorithm is proposed. Firstly, special encoding and decoding methods are used according to the characteristics of the problem; then, introducing differential evolution mutation in the mutation process to improve the diversity of the population; at the same time, during the crossover process, in order to ensure the validity of the solution, the group was divided into two groups for pair wise crossover pairing using the random crossover method. Finally, aiming at the actual operation problems of enterprises, the dynamic scheduling in the event of a machine failure and the dynamic scheduling in the case of urgent orders are discussed separately. The test results show that the method proposed in this paper can get a good scheduling scheme and is an effective method for solving dynamic flexible job shop scheduling problems.**

Keywords—differential evolution, genetic algorithm, flexible job shop, dynamic scheduling

I. INTRODUCTION

With the development of social operation, Job Shop Scheduling Problem (JSP) has become a type of scheduling problem that has attracted much attention. Flexible Job Shop Scheduling Problem (FJSP) is an extension of JSP. Different from JSP, in FJSP, there are multiple processing machines for each operation, and each operation can choose the machine suitable for processing according to the situation. This problem is more complicated than JSP [1]. It belongs to NP-hard problem. But from the perspective of actual operation, this flexible scheduling method is more in line with the needs of actual operation. Therefore, in recent years, many scholars have begun to pay attention to the FJSP. Literature [2] researched on flexible job shop scheduling problem based on improved genetic algorithm, which mainly improved the initialization method of genetic algorithm. Literature [3] used improved intrusion weed algorithm to solve the flexible job shop scheduling problem, in this paper, random key coding and adaptive Gaussian mutation are introduced to improve the effectiveness of the algorithm. In literature [4], flexible job-shop scheduling optimization of discrete manufacturing based on the improved particle swarm optimization algorithm is proposed. This method is mainly to improve the algorithm inertia weight to achieve the purpose of adaptive adjustment. Reference [5] proposed an improved particle swarm algorithm for batch scheduling of flexible job shop. This paper mainly for particle location update mode has been improved. Reference [6] proposed double bare bones particle swarm algorithm for solving flexible job-shop scheduling problem. This method is based on the von Neumann topology and uses the population exchange mechanism to co-evolve the two populations of the backbone particle group and the backbone particle group based on chaotic mutation. This approach achieves a balance between global search and local development. These methods have achieved certain results, and make the research of flexible job shop scheduling a step forward to actual operation. But in the actual operation process, there will also be some disturbances of emergencies, such as: machine failure, urgent order, etc. When these situations occur, it is necessary to readjust the predetermined scheduling scheme to adapt to the new operation environment. Holloway and Nelson first proposed the dynamic scheduling problem in 1974 [7]. At present, the dynamic scheduling problem has become a major research hotpot in the field of workshop scheduling. Flexible job shop rescheduling optimization method for energy-saving based on dynamic events is proposed in [8]. Dynamic strategy-based differential evolution for flexible job shop scheduling optimization is proposed in [9]. Research on dynamic flexible job shop scheduling problem for energy consumption is proposed in [10], etc. From the existing literature, it can be seen that the current research on dynamic scheduling is in the hot research stage.

Aiming at the flexible and dynamic development trend of workshop scheduling problems, this paper studies the Dynamic Flexible Job Shop Scheduling Problem (DFJSP), and proposes multi-step genetic algorithm (GA) for solving dynamic flexible job shop scheduling problem (MGA-DFJSP). Discussed periodic rescheduling, urgent order rescheduling and machine fault rescheduling separately. And improved the basic GA: Firstly, the mutation operation of differential evolution is used to enhance the diversity of the solution. At the same time, to ensure the validity of the solution, a special decoding method is used for the mutated individuals; secondly, during the crossover process, the random crossover method is used to divide the group into two groups for pair wise cross-matching. In the selection process, competing into the next generation by pairing randomly, this method has the ability to retain the best individual. Finally, the actual enterprise operation case was tested. The test results show the effectiveness of the proposed algorithm, which is an effective method for solving dynamic flexible job shop scheduling problems.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.

II. DFJSP PROBLEM MODEL

A. Symbol description

M: Total number of machines available

N: Total number of jobs;

p_i: Operation number of job i;

O_{ij}: Operation j of job i;

T_{ijk}: Processing time on machine k of operation j of job i;

S_{ijk}: Starting time on machine k of operation j of job i;

C_{ijk}: End time on machine k of operation j of job i;

C_i: Make-span of each job;

$$X_{ijk} = \begin{cases} 1, & \text{operation j processed by machine } k \\ 0, & otherwise \end{cases}$$

T_c: Rescheduling cycle;

T_p: The start time of urgent orders;

T_f: The start time of machine malfunction;

T_m: The end time of machine fault maintenance;

B. DFJSP problem description

The DFJSP problems studied in this article are described as follow:

Suppose there are m machines and n jobs in a workshop at the initial moment. Each job contains several operations, and the processing sequence of the job is predetermined. Each operation can be processed in multiple machines and the processing time of each operation is different. The scheduling goal is to select the most suitable machine for each operation, and determine the processing order and start time of each operation of each job on each machine, so that the make-span of the entire system is optimal. In the specific implementation process, once an emergency occurs, the scheduling scheme determined in advance will be disturbed, and then the relevant dynamic scheduling strategy needs to be used for rescheduling. Pre-scheduling is a static scheduling method that assumes that all factors are fixed during the operation process, and pre-scheduling is a correction process that is adopted when interference occurs during the execution of the pre-scheduling. The main difference between rescheduling and pre-scheduling is that the machines are in a different state. Due to the continuity of job processing, some machines may be processing job and being busy during rescheduling.

The three issues will be studied in this article:

(1) Periodic rescheduling: The periodic rescheduling in this paper refers to the re-optimized scheduling of the remaining unfinished jobs and operations at regular intervals (or finished some operations) according to a preset processing plan;

(2) Urgent order: Refers to the emergence of emergency jobs during processing and needs to be rescheduled. At this time, the rescheduled objects include newly arrived jobs, jobs that have been scheduled but have not yet begun processing, and some processes that have started but have not yet completed. When a dynamic event of an emergency purchase order occurs, the jobs at the initial moment are divided into three categories: those that have not yet begun processing (including newly arrived ones), those that have been processed but not yet completed, and those that have been completed. For jobs that have not yet begun processing, add all operations corresponding to the jobs to rescheduling and re-select the processing route; for jobs that have been processed but not yet completed, processing is continuing, and the remaining operations that have not yet begun are rescheduling.

(3) Machine malfunction: When the machine fails, determine the free time of each machine immediately. There are three types of free time: the free time of the faulty machine is the length time from the malfunction start to the end of repair; the free time of the machine being processed is the completion time of the operation being processed; the free time of the free machine refer to the began time of rescheduling. When rescheduling, except for the free time of the faulty machine, the rest enter rescheduling.

Based on the above discussion, we can write the following model for our dynamic scheduling problem:

$$f = min\{\max_{1 \le i \le N}(C_i)\} \tag{1}$$

That is, the maximum work-span of all jobs must be reached to a minimum.

s.t.

$$C_{i,j} = S_{i,j} + \sum_{k=1}^{M} x_{ijk} . t_{ijk},$$

$$(i = 1,2,\ldots N, j = 1,2,\ldots p_i) \tag{2}$$

$$C_i = C_{i,p_i} \tag{3}$$

$$S_{i,(j+1)} \geq C_{i,j} \tag{4}$$

$$\sum_{k=1}^{M} x_{ijk} = 1 \tag{5}$$

$$S_{ijk} \geq C_{rlk} \tag{6}$$

Constraints (2) indicate work-span of the job is the sum of the start processing time and the actual processing time; Constraints (3) indicate the maximum completion time of the job is the completion time of the last operation of the job; Constraints (4) indicate the procedures of the same job have sequence requirements; Constraints (5) mean that an operation can only be processed on one machine and can only be processed once; Constraints (6) indicate a machine can only process one operation at a time. Where C_{rlk} indicate the completion time of the previous job on machine

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.

k.

III. Genetic Algorithm for Solving Dynamic Flexible Job Shop Scheduling Problem

Genetic algorithm (GA) was first proposed by professor Holland of the University of Michigan in the United States. It is a computational model that simulates the natural selection and genetic mechanism of Darwin's biological evolution. It is a search model that simulates natural evolution. It is a method of searching the optimal solution by simulating the natural evolution process. Its characteristic is that it does not require any prior knowledge of the problem to encode parameters, and it is widely used in machine learning, parallel computing, neural network optimization and combination optimization. But like other evolutionary algorithms, GA also falls into a local optimal situation. Based on the characteristics of the scheduling problem, we make the following improvements to the GA.

A. Encoding and decoding

According to the needs of solving DFJSP problem, special encoding and decoding methods are set. Firstly, the encoding of DFJSP is divided into three parts: operation encoding vector, machine encoding vector and time encoding vector. The dimension of the coding vector of the operation is the sum of the total length of all the jobs' operations. Since the machine that can be selected and processing time are known, once the operation code is determined, then select the machine code that can be processed according to the operation code, and the corresponding time code is determined at this time. In the evolution process, only operation codes are involved in mutation and cross operations, machine coding and time coding do not need to participate in the evolution operation, but use a greedy search method to find the corresponding machine number for each operation, and when selecting a machine, always select the machine with the earliest end time and the least processing time. Now take 3 jobs and 3 machines as examples to give specific encoding and decoding methods, the operation number, processing time and available machines for each job are shown in table I:

TABLE I PROCESSING TIME OF 3 JOBS AND 3 MACHINES

jobs	operation	machine		
		1	2	3
J_1	O_{11}	3	2	1
	O_{12}	--	1	3
J_2	O_{21}	2	--	1
	O_{22}	3	4	2
	O_{23}	4	2	1
J_3	O_{31}	--	3	4
	O_{32}	2	3	1

There are 7 operations in total for the 3 jobs in the table above.

The corresponding process coding method is:

1	1	2	2	2	3	3

When the operation' code adopts the mutation operation of differential evolution (DE), a new individual is generated as follows:

0.35	0.87	1.97	1.57	0.26	1.45	1.89

Decoding method:

Sort the newly generated individual codes in ascending order and record the positions of the sorted components in the original individuals, then place the components in the corresponding positions of the individuals before the mutation in the sorted positions. The ascending individuals and their corresponding positions are as follows:

0.26	0.35	0.87	1.45	1.57	1.89	1.97
5	1	2	6	4	7	3

The decoded individuals are:

2	1	1	3	2	3	2

This decoding method guarantees the validity of the individual after the mutation operation.

B. Improvement of the mutation method

Using the DE mutation operation, combining the characteristics of the DE / rand / 1 / bin mutation method and the DE / best / 1 / bin mutation method, a new mutation method is proposed. The mutation formula is:

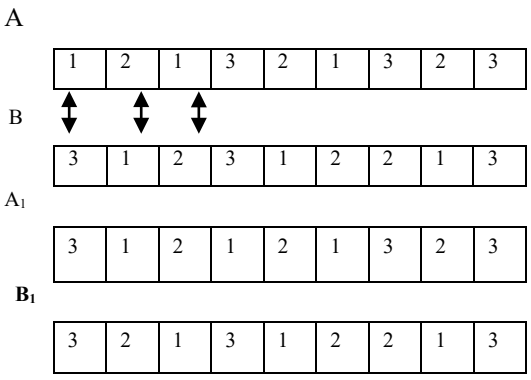
$$v_{ij}(t+1) = \lambda x_{ij}(t) + (1-\lambda) x_{best,j}(t) + F(x_{p1,j}(t) - x_{p2,j}(t)) \quad (7)$$

Where $\lambda \in [0,1]$, $\lambda = \frac{T-t}{T}$, t is the current iterations, T is the maximum iterations, $p1, p2$ are arbitrary random integer from [1, M], F is the mutation factor, and x_{best} is the best individual in the current generation.

C. Improvement of crossover method

This paper uses a random bit crossing method, that is, the new population after mutation is divided into two groups for pairing. For each pair of individuals, A and B, a crossing bit r ($1 \le r \le M$) is randomly selected. The first r genes of an individual are exchanged. In order to ensure the validity of the individual, the following method is used: For example, to exchange A (j) and B (j), look for the position k of B (j) in A, and change A (j) and A (k), which is actually the internal gene exchange of the individual and guarantees the effectiveness of the individual. For example: take 3 jobs and 9 operations as an example, A and B are the coded individuals before crossing. If r = 3, then the first three elements of A and B are exchanged, because the first element of B is 3, which corresponds to the position of the first 3 in A, so the actual exchange is the first and the third element of A, the remaining two elements are exchanged in the same way. The new individuals A_1 and B_1 are shown below:

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.



It can be seen that the crossover of A is actually the exchange between the individual of A. After all the individuals in A are crossed, the same crossover is also performed in B.

IV. Flow Charts of GA for Dynamic Flexible Job Shop Scheduling Problem

Before the dynamic event occurs, the jobs and machine in the workshop are known and will not change. Processing will be performed according to the original plan until the workshop jobs are completely completed. If a dynamic event occurs, it is necessary to re-determine the information of the jobs and the available machine to carry out rescheduling. The processing route selection, machine selection, and the rescheduling scheme are optimized at the same time. Based on this, a DFJSP process framework based on GA is proposed, as shown in Figure 1:

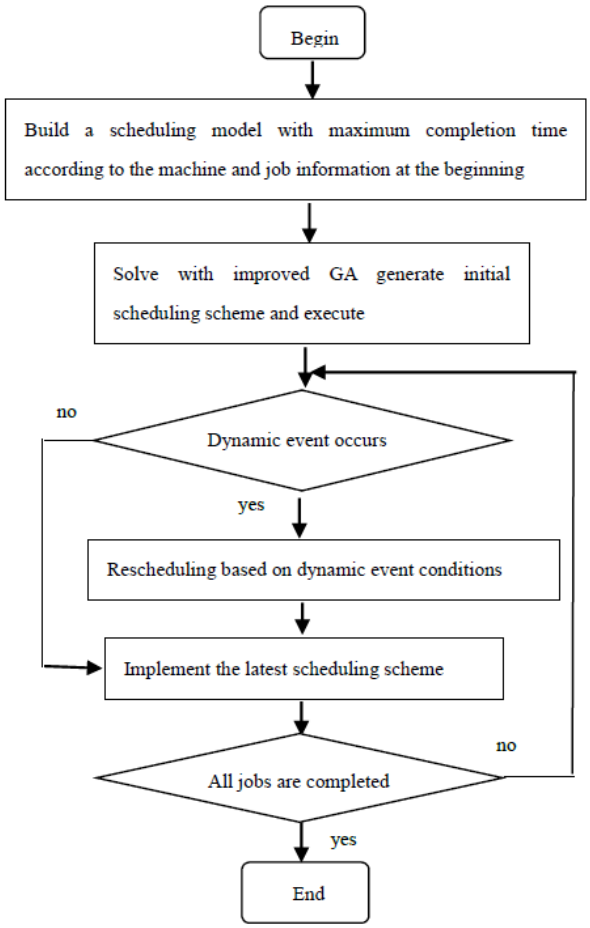


Fig.1.Dynamic flexible job shop scheduling process framework based on GA

The specific steps for solving the DFJSP as follow:

- Step1 Set the population size M, maximum iterations$T$, cross probabilityP_c, initial iteration$t = 0$, set formula (1) as the fitness function, randomly generates the initial code of the operation, and executes the initial scheduling scheme;
- Step2 Determine whether a dynamic event has occurred. If so, go to step 3. If not, go to step 4.
- Step3 Determine the type of dynamic events, determine the rescheduled object, and execute the latest scheduling scheme according to the rescheduled strategy;
- Step4 Perform mutation operation according to 3.2 and decode operation according to 3.1;
- Step5 Perform cross operation according to 2.3;
- Step6 Calculate fitness and keep the optimal solution;
- Step7 Determine whether all the jobs in the workshop are completed. If so, end the process and output the optimal solution. Otherwise, set $t = t + 1$ and go to step 2.

V. The Simulation Results and Analysis

In order to test the effectiveness of the algorithm, the flexible job shop scheduling situation of the actual production company was calculated. This case is an example of the production of 5 operations and 6 machines [11], this article separately discusses periodic rescheduling, dynamic scheduling when machine failure, and dynamic scheduling when urgent order arrival. Programming by MATLAB, the maximum iteration is 50 and the population size is 50. Cross probability $P_c = 0.75$, mutation probability$F = 0.3$. The algorithm runs 10 times. Fig.2, Fig.3 and Fig.4 show the scheduling Gantt chart of the optimal simulation results.

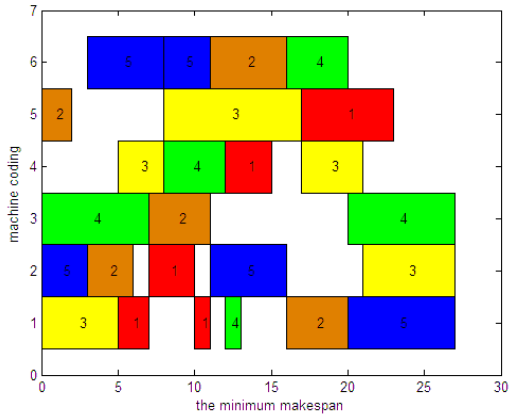


Fig.2. Gantt chart of periodic rescheduling

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.

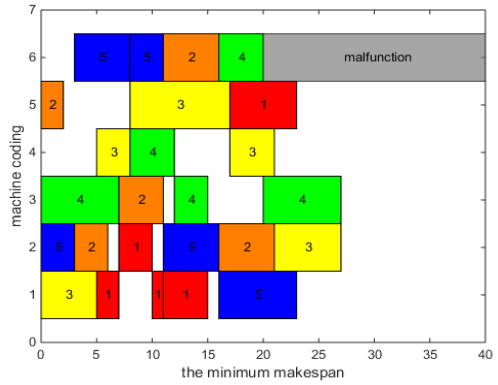


Fig.3. Gantt chart of one machine faults

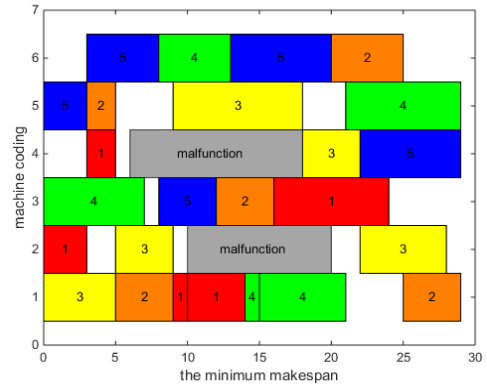


Fig.4.Gantt chart of multiple machines fault

Fig.2 is a Gantt chart of periodic rescheduling, that is, it is rescheduled every time a predetermined number of operations are completed, and the minimum make span is 27; for machine failure condition, this paper considers the combination of periodic rescheduling and fault scheduling, whichever comes first executes the corresponding command first. Due to the random nature of machine failures, every machine is likely to fail at every moment. For comparison with the literature [11], when discussing a machine failure, choose the same settings as the literature. Set the machine No. 6 to fail at time 20, and the failure time is 30. First perform periodic rescheduling; when reached 20, the fault rescheduling is performed immediately, during the 20-30 time period, the machine is free. After rescheduling, the operations on the failed machine are arranged for production on other machines. The final optimal completion time is 27. Fig.3 shows the Gantt chart for scheduling. It is the best result of running 10 times by MGA-DFJSP. This article also considers the failure of multiple machines at the same time. For example: machine No. 4 malfunctioned at 6 o'clock until 18 o'clock, machine No. 2 malfunctioned at 10 o'clock until 20 o'clock. At 6 o'clock, the remaining operations need to be rescheduled once, at 6 o'clock, due to machine 2 failure, rescheduling is needed. At 18 o'clock, the fault of machine No. 4 was eliminated and the remaining jobs could be processed. At this time, the remaining jobs need to be rescheduled. Fig.4 shows the Gantt chart for the scheduling of multiple machines. It can be seen from the figure that the machine 3 is scheduled to be processed immediately after the fault of machine 4 is eliminated. Due to the use of a real-time judgment plan, in this case, the optimal completion time is not affected much, and the completion time is 29. Table II shows the comparison results of the algorithm and the literature algorithm.

TABLE II COMPARISON OF DIFFERENT SCHEDULING SCHEMES

scheduling / time	enterprise actual scheduling	literature algorithm	MGA-DFJSP
downtime	20	20	20
machine maintenance time	30	0	0
ideal completion time	37	37	27
actual completion time	67	37	27

By comparing the scheduling results in Table II, we can find that using the dynamic scheduling method in this paper, the combination of periodic scheduling and fault scheduling, optimize the scheduling every once in a while, before failure, and use rescheduling immediately after the failure of the machine. The job on the failed machine can be directly inserted into other processional machines for processing without waiting for the repair of the failed machine. The MGA-DFJSP is 10 times shorter than that in the literature [11], and is 60% shorter than the actual completion time of the enterprise, thereby greatly reducing the actual completion time. In order to further verify the effectiveness of the MGA-DFJSP, a dynamic scheduling situation of urgent orders is designed. Because in the production process, the sudden emergency purchase order is also random, so it may be a good idea to assume that there is an emergency purchase order at time 5 (the job number is 6, 3 operations). At this time, check the processing status of the job, cancel the corresponding unprocessed job on the machine according to the optional processing machine of the emergency job, give priority to the emergency job, and reschedule the remaining jobs. Fig.5 shows the corresponding scheduling Gantt chart. As can be seen from the Fig.5, the job 6 is added at 5 to give priority to production, the operations originally arranged on the corresponding machines are moved backwards.

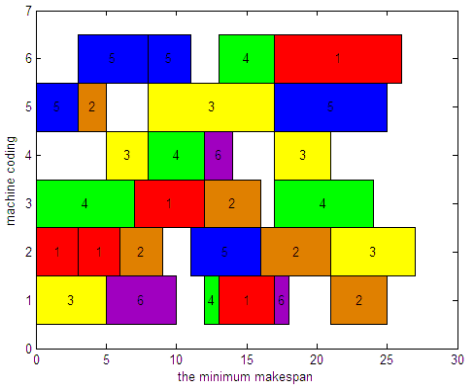


Fig.5.Gantt chart of urgent order scheduling

VI. CONCLUSIONS AND DIRECTIONS FOR FURTHER RESEARCH

The GA borrows from the evolutionary quasi-sides of "natural selection, survival of the fittest", has the characteristics of implicit parallelism and global solution space search, and is widely used in the field of production scheduling. However, as with most optimization algorithms, there are also disadvantages such as being easy to premature

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.

and being limited to local excellence, and its performance needs to be further improved. This paper improves the GA, proposes a multi-step GA, and uses the improved GA to solve the DFJSP. The simulation results show that the improved algorithm has a significant improvement over the compared algorithms in solving dynamic scheduling problems based on machine faults and emergency orders, and it is an effective method for solving DFJSP.

The next work is to further optimize the algorithm to solve the distributed dynamic scheduling problem.

ACKNOWLEDGMENTS

This work has been financed by: Science and Technology Research Project of Guangxi universities (KY2015YB521); Youth Education Teachers' Basic Research Ability Enhancement Project of Guangxi universities (2019KY1098); Key Project in Teaching Reform of Lushan College of Guangxi University Science and Technology (2018JGZ004).

REFERENCES

[1] Y.W. An, H.S. Yan, "Solution Strategy of Integrated Optimization of Production Planning and scheduling in a Flexible Job-shop," Acts Automatic a Sinic, 2013, vol. 39, no. 9, pp. 1476-1491(in Chinese).

[2] R. Cao, X. Hou, P, et.al., "Research on Flexible Job Shop Scheduling Problem Based on Improved Genetic Algorithm," Computer &Digital Engineering, 2019, vol. 42, no. 2, pp. 285-288.

[3] X. Zhang, K. Li, et.al. "Improved Intrusion Weed Algorithm for Solving Flexible Job Shop Scheduling Problem," Journal of System Simulation, 2018, vol. 30, no. 11, pp. 4467-4475.

[4] S.W. Li, C.L. Zhang, Z.J. Zhou, "Flexible Job-shop Scheduling Optimization of Discrete Manufacturing Based on the Improved Particle Swarm Optimization Algorithm," Modular Machine Tool & Automatic Manufacturing Technique, 2018, vol. 11, pp. 150-152.

[5] J. Zhang, W.L. Wang, et.al., Improved particle swarm algorithm for batch splitting flexible job shop scheduling," Control and Decision, 2012, vol. 27, no. 4, pp. 513-518.

[6] Y.M. Dai, M.H. Wang, et.al., "Double Bare Bones Particle Swarm Algorithm for Solving Flexible Job-shop Scheduling Problem," Journal of System Simulation, 2017, vol. 29, no. 6, pp. 1268-1275.

[7] C.A. Holloway, R.T. Nelson, "Job shop scheduling with due dates and variable processing times," Management Science, 1974, vol. 20, pp. 1264-1275.

[8] "Flexible job shop rescheduling optimization method for energy-saving based on dynamic events," Computer Integrated Manufacturing Systems, 2019, 4: HTTP://kns.cnki.net/kcms/detail/11.5946.tp.20190408.0954.003.HTML.

[9] G.J. Zhang, W. Wang, "Dynamic Strategy-based Differential Evolution for Flexible Job Shop Scheduling Optimization," Computer Science, 2018, vol. 45, no. 10, pp. 240-244.

[10] C. Chen, Y. Wang, "Research on Dynamic Flexible Job Shop Scheduling Problem for Energy Consumption," Journal of System Simulation, 2019, vol. 29, no. 9, pp. 2168-2174.

[11] Z.J. Wu, H.Y. He, et.al., "Flexible Job Shop Dynamic Scheduling Problem Research with Machine Fault," Machine Design and Research, 2015, vol. 31, no. 3, pp. 95-98.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 17,2026 at 06:28:34 UTC from IEEE Xplore. Restrictions apply.